Reusable Mathematical Models for Fusion Diagnostics

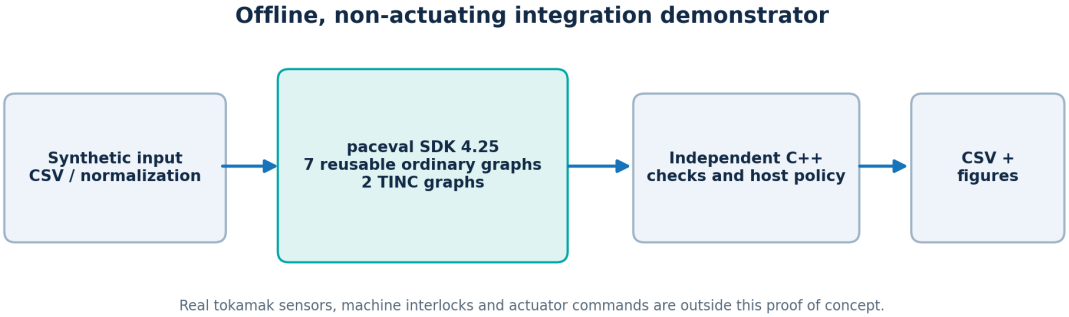
Scientific implementation report | paceval UG | 10 November 2026
Expanded C++ feasibility demonstrator using the original paceval API 4.25

Research result. A functioning offline C++ demonstrator creates **seven reusable paceval computation graphs and two additional TINC-enabled graphs** for a defined family of **synthetic, normalized diagnostic and monitoring expressions**. Using the original paceval Linux runtime, 256 synthetic inputs produced 1,792 tested mathematical outputs without configured tolerance violations. A separate multiprecision test confirmed all 24 selected TINC pointwise inclusions. This is a demonstrator of mathematical integration and reproducibility; it is not a calibrated tokamak diagnostic or a real-time machine-control system.

1. Scientific motivation and precise scope

Magnetic-confinement experiments employ complex plasma models, sensor interpretation, operational boundaries and safety mechanisms. Published work on the TCV tokamak illustrates the sophistication of such control systems [1,2]. This study instead isolates **explicit mathematical submodels** that can be repeatedly evaluated after input preprocessing. Its question is whether the original paceval mathematical execution engine can implement, reuse and independently validate such expressions through its C/C++ API [3].

The supplied three-page research proposal introduced a polynomial diagnostic response as an **illustrative** benchmark. The present project extends that original expression into a self-contained, inspectable family of seven formulas and two independent interval-enabled graphs. Its input readings, coefficients, model variation, monitoring envelope and advisory result are all synthetic and dimensionless. The models are not fitted to or validated against fusion-machine measurements.



A deliberately isolated offline mathematical kernel. The existing machine control system is not part of this implementation.

Integration boundary: The host owns data acquisition, normalization, policy checks, sensor validity, interlocks, scheduling and any actuator system. The demonstrator implements none of these live plant interfaces. paceval evaluates explicit formulas, with the C++ host performing error handling, CSV storage and independent numerical tests. A future fusion research partner must provide any physically meaningful functions, approved datasets, uncertainty models and machine-specific constraints.

2. Implemented mathematical research models

Let `i` and `b` be bounded, normalized current-like and field-like inputs. `a0...a4` and `b0...b2` are synthetic calibration coefficients. Other coefficients are listed in the synthetic input table. All results below are dimensionless and are interpreted **only as mathematical quantities**.

Polynomial diagnostic response (graph 1):

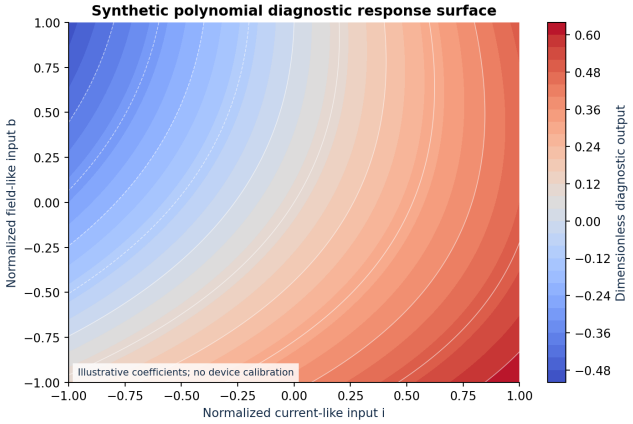
```
y(i,b) = a0 + a1*i + a2*b + a3*i*b + a4*b^2
```

This preserves the core expression proposed in the original paper, while evaluating it over changing synthetic coefficients and input samples.

Nonlinear alternate response (graph 2):

```
z(i,b) = b0 + b1*tanh(k*i) + b2*exp(-c*b^2)
```

The second graph tests a different expression family, not a learned or validated tokamak surrogate. Its purpose is to show that paceval handles nonlinear analytic terms using the same reusable evaluation interface.



Synthetic polynomial output as a function of normalized inputs. Coefficients are illustrative, not machine-calibrated.

Analytic sensitivity graphs (graphs 3–4):

```
dy/di = a1 + a3*b
dy/db = a2 + a3*i + 2*a4*b
```

The two graph expressions are checked against independently calculated central finite differences of the original polynomial.

First-order uncertainty expression (graph 5): For input standard deviations `s_i` and `s_b`, and **assuming small, independent perturbations**, the demonstrator evaluates

```
sigma_y = sqrt((dy/di)^2 * si^2 + (dy/db)^2 * sb^2)
```

This is a linearized input-noise propagation model; it does not represent comprehensive fusion diagnostic uncertainty and omits correlated input errors.

Synthetic envelope margin (graph 6):

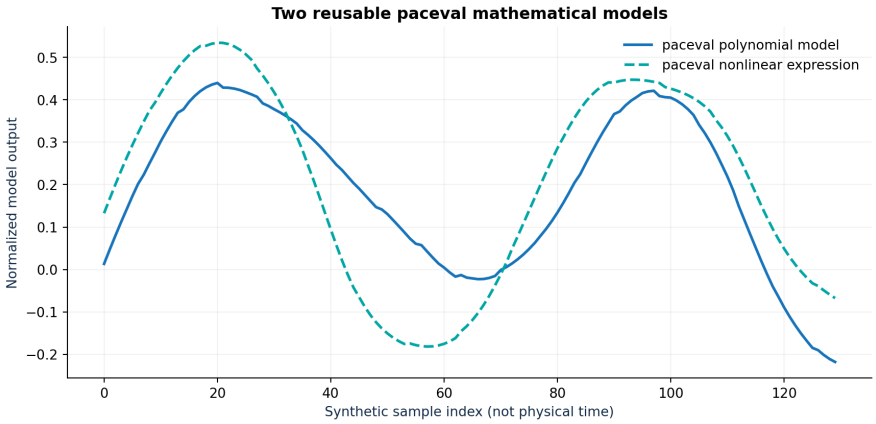
```
m(i,b) = 1 - (i/ilim)^2 - (b/blim)^2
```

The C++ host labels outputs with `m < 0` as outside a **mathematical illustrative envelope**. These are not physical safety interlock limits.

Non-actuating advisory expression (graph 7):

```
u = kp*(target - y(i,b)) - kd*di
```

This simply demonstrates an explicit, repeatable control-related expression. The program does **not** calculate physical coil voltages, provide feedback to an actual machine or claim control stability.



Actual paceval output samples for the polynomial and nonlinear model. The horizontal axis is a sample index, not physical time.

3. Concrete paceval SDK implementation

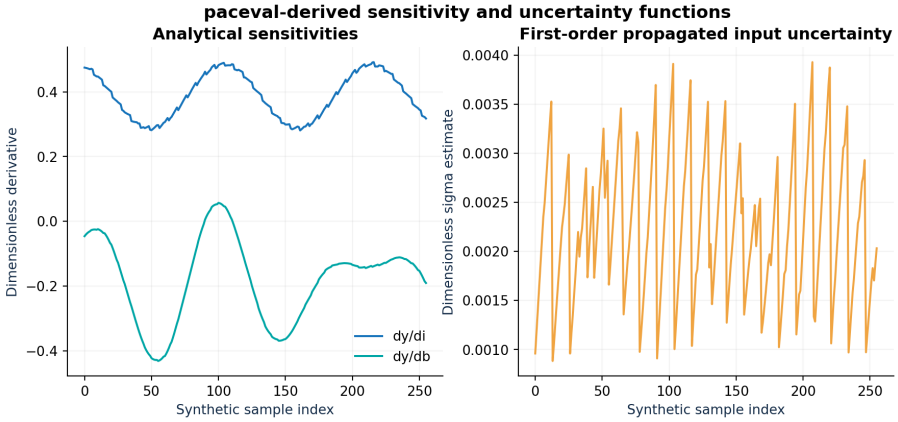
The main C++ source includes the original vendor API header. The repository's CMake project compiles the original paceval API forwarding implementation as a second source file, then links the **externally supplied original paceval Linux shared runtime**. This replaces the need for custom runtime symbol lookup or copied function-pointer declarations.

The actual production source uses the following API calls:

API function	Role in the tested project
`paceval_InitializeLibrary` / `paceval_FreeLibrary`	Engine lifetime
`paceval_CreateComputation` / `paceval_DeleteComputation`	Construction, reuse and release of nine computation objects
`paceval_dGetMultipleComputationsResultsExt`	Grouped evaluation of seven graphs across input batches
`paceval_dGetComputationResult`	Pointwise TINC result and interval endpoint retrieval
`paceval_GetIsError` / `paceval_GetErrorInformation`	Explicit status checking and diagnostics

All nine expressions use one ordered schema of **20 named variables**. The input buffer is sample-major, and the multi-graph result buffer is interpreted in the vendor-documented **graph-major** order. Graph ownership is managed with C++ RAII, ensuring that graph handles are released before the engine is shut down. The tested implementation does not silently substitute native C++ results when paceval returns an error.

The vendor files retain their copyright and AGPL notices. Their line endings/character encoding were made portable and the header include path inside `paceval_main.cpp` was adjusted for the self-contained source folder; no API signatures were replaced. The vendor binary is not distributed in this repository.



Analytic sensitivities and the model's explicitly limited first-order noise calculation.

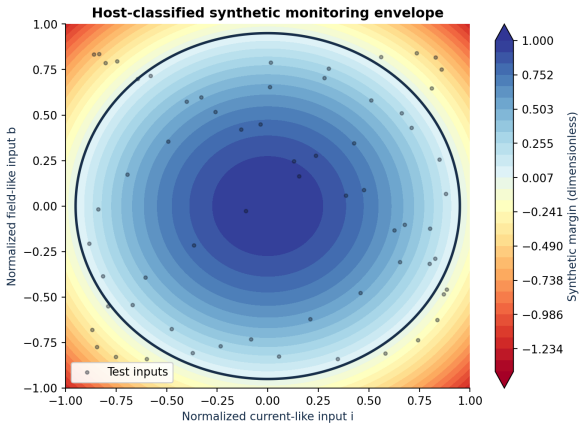
4. Reproducibility and numerical verification

The 256 deterministic synthetic input rows cover varying normalized signals, uncertain input terms, target values and coefficients. The paceval evaluation results preserve seven output quantities per input row. The test suite includes a native mathematical self-test, real-runtime integration test, an independent 85-digit ordinary-function audit, and a separate 110-digit TINC audit.

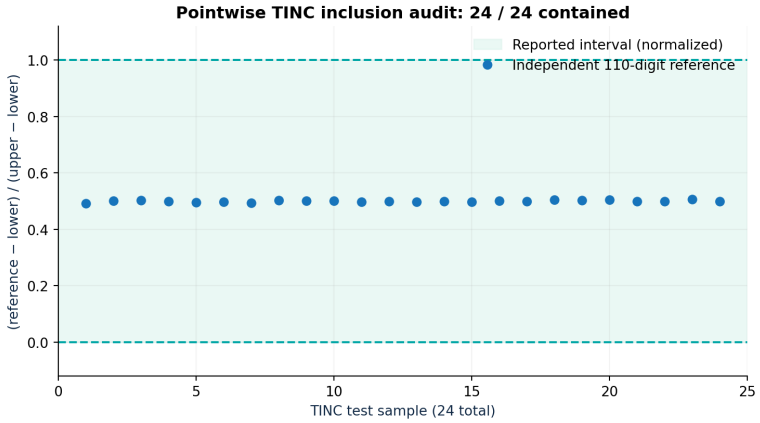
Verification	Result on 10 November 2026
Original paceval runtime and nine prepared computation objects	Successful creation and evaluation
Native C++ vs paceval, 256 × 7 function evaluations	**1,792/1,792 within tolerance**; max observed absolute difference `1.11e-16` in the C++ comparison
Independent 85-decimal-digit evaluation	**1,792/1,792 within tolerance**; max absolute difference about `1.99e-16`
Sensitivity derivatives vs independent central differences	Passed on 65 synthetic native test inputs
24 reported TINC intervals vs independent 110-decimal-digit pointwise reference	**24/24 reference points enclosed**
GCC 14.2 and Clang 17, CMake + CTest	2 of 2 tests passed for each compiler
AddressSanitizer + UndefinedBehaviorSanitizer	2 of 2 tests passed with external-library leak detection disabled

The reported TINC bounds were tested at **24 individual binary64 input points**, 12 for the diagnostic expression and 12 for the illustrative envelope margin. The CSV evidence is independently recomputable with the provided 110-digit script. This test is **not** a

proof that arbitrary paceval TINC intervals rigorously enclose every exact result. Higher-dimensional input uncertainty propagation and certification of a control system are outside the stated experiment.



Illustrative monitoring envelope. This is a mathematical classifier, not a physical tokamak operating boundary.



Independent 110-digit references lie inside each of the 24 tested pointwise paceval TINC intervals.

5. Exact reproducibility path

The build was tested on **Debian GNU/Linux 13**, with **GCC 14.2** and **Clang 17**, and the original paceval SDK API 4.25, using a separately supplied `libpaceval_linux_sharedLIB.so`. The binary's SHA-256 in the tests was `acf63a87df0ee822b467edbca887393d31ec0ea03562e1abf802d2512a5bfb12`. These environment details support reproducibility, not a processor-performance comparison.

From the repository root:

```
export PACEVAL_LIB=/absolute/path/to/libpaceval_linux_sharedLIB.so
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME="$PACEVAL_LIB"
cmake --build build -j
ctest --test-dir build --output-on-failure
./build/fusion --input-csv examples/sensors.csv --samples 256 --batch 32 \
  --output-csv build/evaluations.csv --tinc-csv build/tinc_samples.csv
```

Run the independent mathematical audit and interval audit on the generated CSV files:

```
python3 -m pip install mpmath
python3 src/validate_outputs.py examples/sensors.csv build/evaluations.csv
python3 src/high_precision_reference.py build/tinc_samples.csv
```

The included figure-generation script regenerates the scientific figures from the committed CSV artifacts (with `numpy`, `pandas`, `matplotlib` and `mpmath`). The GitHub README inventories each file and explains the tested API integration. All paths above refer to real repository contents.

6. Research value and next scientific steps

The demonstrated capability is the **separation of reusable explicit mathematics from a higher-level diagnostic workflow**. The project shows a consistent way to prepare formulas once, evaluate grouped mathematical outputs, attach explicit error checks and perform independent numerical verification, while preserving the host’s responsibility for device-specific logic.

The logical next research stage requires input from a fusion laboratory: an approved diagnostic expression with physical units, validated calibration coefficients, a documented sensor-noise model, real or authorized recorded measurements, and explicit expected error behavior. This would permit a scientifically meaningful assessment of the mathematics against laboratory references. Only after offline validation could an isolated, **non-actuating shadow-mode adapter** be considered. Integration into operational safety or physical plasma control would require separate machine-specific engineering, verification and authorization. A possible future FPGA/ASIC implementation is an architectural research question, not a capability tested here.

References and project links

1. Degrave, J. et al. (2022), *Magnetic control of tokamak plasmas through deep reinforcement learning*, Nature 602, 414–419. DOI:10.1038/s41586-021-04301-9.
2. Tracey, B. et al. (2024), *Towards practical reinforcement learning for tokamak magnetic control*, Fusion Engineering and Design 200, 114161. DOI:10.1016/j.fusengdes.2024.114161.

3. paceval UG, paceval C/C++ API documentation, version 4.25 as used in the supplied source files.
4. Project source: src/fusion.cpp; src/CMakeLists.txt; SDK header and implementation.
5. Independent evidence: input samples, evaluated outputs and TINC samples.

paceval UG | Munich, Germany | November 2026 | info@paceval.com

The independent feasibility findings concern defined synthetic mathematical functions. They do not imply collaboration with the cited tokamak experiments or successful deployment in a fusion facility.